Pinocchio and Groth16 notes

arnaucube

2026

Abstract

Notes taken while studying the schemes Pinocchio ([1]) and Groth16 ([2]).

Usually while reading books and papers I take handwritten notes in a notebook, this document contains some of them re-written to $LaTeX$.

The proofs may slightly differ from the ones from the books or papers, since I try to extend them for a deeper understanding.

Contents

1 Introduction

2 Notation

Let $g \in \mathbb{G}$ be the generator of the group $\mathbb{G}$, ie. $\mathbb{G} =< g >$, then denote with $[a]$ by $g^a \in \mathbb{G}$.

3 R1CS

Rank-One Constraint System (R1CS) is the representation of the constraints of the circuit by 3 linear combinations, where there is one constraint per operation.

$$\begin{aligned}
(a_{11}w_1 + a_{12}w_2 + ... + a_{1n}w_n) \cdot (b_{11}w_1 + b_{12}w_2 + ... + b_{1n}w_n) \\
-(c_{11}w_1 + c_{12}w_2 + ... + c_{1n}w_n) = 0 \\
(a_{21}w_1 + a_{22}w_2 + ... + a_{2n}w_n) \cdot (b_{21}w_1 + b_{22}w_2 + ... + b_{2n}w_n) \\
-(c_{21}w_1 + c_{22}w_2 + ... + c_{2n}w_n) = 0 \\
(a_{31}w_1 + a_{32}w_2 + ... + a_{3n}w_n) \cdot (b_{31}w_1 + b_{32}w_2 + ... + b_{3n}w_n) \\
-(c_{31}w_1 + c_{32}w_2 + ... + c_{3n}w_n) = 0 \\
\dots \\
(a_{m1}w_1 + a_{m2}w_2 + ... + a_{mn}w_n) \cdot (b_{m1}w_1 + b_{m2}w_2 + ... + b_{mn}w_n) \\
-(c_{m1}w_1 + c_{m2}w_2 + ... + c_{mn}w_n) = 0
\end{aligned}$$

Where the R1CS linear combinations can be represented by three matrices A, B and C:

$$A = \begin{pmatrix} a_{11} & a_{12} & ... & a_{1n} \\ a_{21} & a_{22} & ... & a_{2n} \\ a_{31} & a_{32} & ... & a_{3n} \\ ... & & & \\ a_{m1} & a_{m2} & ... & a_{mn} \end{pmatrix} B = \begin{pmatrix} b_{11} & b_{12} & ... & b_{1n} \\ b_{21} & b_{22} & ... & b_{2n} \\ b_{31} & b_{32} & ... & b_{3n} \\ ... & & & \\ b_{m1} & b_{m2} & ... & b_{mn} \end{pmatrix}$$

$$C = \begin{pmatrix} c_{11} & c_{12} & ... & c_{1n} \\ c_{21} & c_{22} & ... & c_{2n} \\ c_{31} & c_{32} & ... & c_{3n} \\ ... & & & \\ c_{m1} & c_{m2} & ... & c_{mn} \end{pmatrix}$$

Where $(A, B, C) = Aw \circ Bw - Cw = 0$

The process that converts the *flat constraints* into *R1CS* is the following:

Lets assume that we have a constraint $5w_3 = w_1 \cdot 2w_2$ (being w_1, w_2 and w_3 the signals 1, 2 and 3 respectively)). We can represent this constraint in the matrices form by:

$$A = \begin{pmatrix} 1 & 0 & 0 & ... & 0 \end{pmatrix}$$
$$B = \begin{pmatrix} 0 & 2 & 0 & ... & 0 \end{pmatrix}$$
$$C = \begin{pmatrix} 0 & 0 & 5 & ... & 0 \end{pmatrix}$$

Which in the R1CS equation form would look as:

$$(1 \cdot w_1 + 0 \cdot w_2 + 0 \cdot w_3 + ... + 0 \cdot w_n) \cdot (0 \cdot w_1 + 2 \cdot w_2 + 0 \cdot w_3 + ... + 0 \cdot w_n) - (0 \cdot w_1 + 0 \cdot w_2 + 5 \cdot w_3 + ... + 0 \cdot w_n) = 0$$

If we simplify the previous liniar combination we get:

$$\begin{aligned} (1 \cdot w_1) \cdot (2 \cdot w_2) - (5 \cdot w_3) &= 0 \\ w_1 \cdot 2w_1 - 5w_3 &= 0 \\ 5w_3 &= w_1 \cdot 2w_1 \end{aligned}$$

Where $5w_3 = w_1 \cdot 2w_1$ is the constraint that we wanted to represent in the R1CS format.

The nice thing from R1CS is that allows us to standaridze the constraints in an equation fashion, where all the signals are involved and we 'activate' these ones that are used in each constraint. Since we have now the constraints in a polynomial format, we can do work with it in the polynomial way.

4 QAP (Quadratic Arithmetic Programs)

Once we have the constraints of a circuit represented in *R1CS*, we can do one more iteration and represent it in *Quadratic Arithmetic Programs (QAP)*. *QAP* is a representation of the XXX in a linear combination of the *R1CS*, represented by 3 polynomials $a(X)$, $b(X)$ and $c(X)$. The *QAP* consists of the 3 polynomials $a(X)$, $b(X)$ and $c(X)$, plus a divisor polynomial $h(X) \in \mathbb{F}[X]^{<d}$.

The main idea behind the *QAP* is that they 'contain' all the constraints of the circuit, meaning that these polynomials need to be able to generate all the constraints.

In order to 'collapse' the *R1CS* polynomials into the 3 polynomials of the *QAP*, we make use of *polynomial interpolation*, also named *Lagrange interpolation*. *Lagrange interpolation* allows for a group of points, to find the smallest degree and unique polynomial that goes through all those points. It can be calculated by computing:

$$L(X) = \sum_{j=0}^{n} y_j l_j(X)$$

where $l_j(X)$ is:

$$l_j(X) = \prod_{0 \leq m \leq k; m \neq j} \frac{X - x_m}{x_j - x_m} ... \frac{(X - x_{j-1})}{(x_j - x_{j-1})} \frac{(X - x_{j+1})}{(x_j - x_{j+1})} ... \frac{X - x_k}{x_j - x_k}$$

The intuition behind the *Lagrange interpolation* is quite simple. If we have two points (on a 2 dimensional plane) we can find a line that goes through both points (this can be done with a pen and a ruler on a paper), if we have 3 points we can find a curved line that goes through all the three points, if we do this through math trying to get the lowest degree curve we obtain a 2nd degree polynomial. This can be generalized using the *Lagrange interpolation*, allowing us for a group of n points to find a $n-1$ degree polynomial that goes through all of them.

In our case, *Lagrange interpolation* is used to go from the *R1CS* polynomials into the *QAP* polynomials, which are defined by:

$$a(X) = (a_1(X)w_1 + a_2(X)w_2 + ... + a_n(X)w_n) = \sum_{i=1}^{n} a_i(X)w_i$$

$$b(X) = (b_1(X)w_1 + b_2(X)w_2 + ... + b_n(X)w_n) = \sum_{i=1}^{n} b_i(X)w_i$$

$$c(X) = (c_1(X)w_1 + c_2(X)w_2 + ... + c_n(X)w_n) = \sum_{i=1}^{n} c_i(X)w_i$$

The polynomials $a(X), b(X), c(X)$ are the QAP, such that

$$a(X) \cdot b(X) = c(X)$$

so that

$$p(X) = a(X) \cdot b(X) - c(X) = 0$$

that is,

$$\begin{aligned} p(X) =& (a_1(X)w_1 + a_2(X)w_2 + ... + a_n(X)w_n) \\ & \cdot (b_1(X)w_1 + b_2(X)w_2 + ... + b_n(X)w_n) \\ & - (c_1(X)w_1 + c_2(X)w_2 + ... + c_n(X)w_n) \\ =& (\sum_{i=1}^{n} a_i(X)w_i) \cdot (\sum_{i=1}^{n} b_i(X)w_i) - (\sum_{i=1}^{n} c_i(X)w_i) = 0 \end{aligned}$$

The main idea, is that $p(X)$ defines the constraints of our circuit, so that when evaluating $p(X)$ at the challenge, it will be equal to zero.

5 QAP Relation

Want to ensure that

$$(\sum_{i=1}^{n} a_i(X)w_i) \cdot (\sum_{i=1}^{n} b_i(X)w_i) - (\sum_{i=1}^{n} c_i(X)w_i) = 0$$

for a given witness $(w_1, \ldots, w_n) \in \mathbb{F}^n$, which we divide between public inputs and private inputs:

$$\begin{aligned} \text{public inputs: } & \phi = (w_1, \ldots, w_l) \in \mathbb{F}^l, \\ \text{private inputs: } & w = (w_{l+1}, \ldots, w_n) \in \mathbb{F}^{n-l}, \end{aligned}$$

We denote the relation

$$R_{QAP} = \begin{cases} (\phi, w) \;\Big|\; \phi = (w_1, \ldots, w_l) \in \mathbb{F}^l, \\ \qquad w = (w_{l+1}, \ldots, w_n) \in \mathbb{F}^{n-l}, \\ \qquad (\sum_{i=1}^{n} a_i(X) w_i) \cdot (\sum_{i=1}^{n} b_i(X) w_i) \equiv \sum_{i=1}^{n} c_i(X) w_i \mod Z(X) \end{cases}$$

where $Z(X)$ is the "zero polynomial" or "vanishing polynomial".
Set

$$a(X) = \sum_{i=1}^{n} a_i(X) w_i, \quad b(X) = \sum_{i=1}^{n} b_i(X) w_i), \quad c(X) = \sum_{i=1}^{n} c_i(X) w_i$$

so that

$$a(X) \cdot b(X) \equiv c(X) \bmod Z(X)$$

which means that we can prove that

$$\exists\, h(X) \in \mathbb{F}[X]^{<d} \text{ s.th. } a(X) \cdot b(X) - c(X) = h(X) \cdot Z(X)$$

6 Simplified protocol

Protocol to prove $a(X)b(X) - c(X) = 0$ in order to provide intuition on what we're trying to do.

Setup. The protocol requires a secret random value, which it is encrypted over the elliptic curve group. That is, for given $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$ generators of curves $\mathbb{G}_1$ and $\mathbb{G}_2$ respectively, set $[\tau]_1 = g_1^{\tau}$ and $[\tau]_2 = g_2^{\tau}$.

$$\begin{aligned} \{[\tau^i]_1\}_0^n &= ([\tau^0]_1, [\tau^1]_1, [\tau^2]_1, \ldots, [\tau^{n-1}]_1) \\ \{[\tau^i]_2\}_0^n &= ([\tau^0]_2, [\tau^1]_2, [\tau^2]_2, \ldots, [\tau^{n-1}]_2) \end{aligned}$$

Prover.

i. commit to $a(X), b(X), c(X) \in \mathbb{F}[X]$

$$cm(a) = [a(\tau)]_1 = \sum_{i=0}^{\delta a} a_i \cdot [\tau^i]_1 \in \mathbb{G}_1$$

notice that it is equal to $\sum a_i \cdot g_1^{\tau^i}$ without knowing τ.
Same is done for $b(X)$ and $c(X)$:

$$cm(b) = [b(\tau)]_2 = \sum_{i=0}^{\delta b} b_i \cdot [\tau^i]_2 \in \mathbb{G}_2$$

$$cm(c) = [c(\tau)]_1 = \sum_{i=0}^{\delta c} c_i \cdot [\tau^i]_1 \in \mathbb{G}_1$$

For a challenge $z \in^R \mathbb{F}$, set by the verifier, open the commitments:

$$q_a(X) = \frac{a(X) - a(z)}{X - z}$$

Since $a(X) - \underbrace{a(z)}_{=y}$ should be zero at $X - z$, so $a(X) - a(z)$ has a root at z, hence it is divisible by $(X - z)$, therefore,

$$\exists\ q_a(X) \ \text{ such that } a(X) - a(z) = (X - z) \cdot q_a(X)$$

therefore

$$q_a(X) = \frac{a(X) - a(z)}{X - z}$$

Same is done for $b(X)$ and $c(X)$,

$$q_b(X) = \frac{b(X) - b(z)}{X - z}$$

$$q_c(X) = \frac{c(X) - c(z)}{X - z}$$

And then, evaluate them at the encrypted τ:

$$[q_a(\tau)]_1 = \sum_{i=0}^{\delta q_a} q_{a_i} \cdot [\tau^i]_1 \in \mathbb{G}_1$$

similarly for $[q_b(\tau)]_2 \in \mathbb{G}_2$ and $[q_c(\tau)]_1 \in \mathbb{G}_1$.

ii. for the relation that we want to prove, $a(X) \cdot b(X) - c(X) = 0$,

$$\exists\ q(X) \in \mathbb{F}[X] \ \text{ s.th. } \ a(X)b(X) - c(X) = q(X) \cdot z(X)$$

So the prover computes

$$q(X) = \frac{a(X) \cdot b(X) - c(X)}{z(X)}$$

and it's encrypted evaluation

$$[q(\tau)]_1 = \sum_{i=0}^{\delta q} q_i \cdot [\tau^i]_1 \in \mathbb{G}_1$$

Verifier.

i. verify the commitments:
The prover has sent the commitments $[a(\tau)]_1$, $[b(\tau)]_2$, $[c(\tau)]_1$, and their respective evaluations at the challenge $z \in \mathbb{F}$: $a(z), b(z), c(z) \in \mathbb{F}$; together with their respective opening proofs: $[q_a(\tau)]_1, [q_b(\tau)]_2, [q_c(\tau)]_1$

For each commitment, the verifier checks:

$$e([q_a(\tau)]_1, [\tau]_2 - [z]_2) \stackrel{?}{=} e([a(\tau)]_1 - [a(z)]_1, [1]_2)$$

which is a way of checking the existence of $q_a(X)$ such that $q_a(X) \cdot (X - z) = a(X) - a(z)$.

If the checks passes, it means that $q_a \in \mathbb{F}[X]$ exists, and thus $a(X) - a(z)$ is divisible ("vanishes") at $(X - z)$, hence $a(X) - a(z)$ has a root at z.

Same is done for $b(X)$ and $c(X)$ commitments:

$$e([\tau]_1 - [z]_2, [q_b(\tau)]_1) \stackrel{?}{=} e([1]_1, [b(\tau)]_1 - [b(z)]_1)$$

$$e([q_c(\tau)]_1, [\tau]_2 - [z]_2) \stackrel{?}{=} e([c(\tau)]_1 - [c(z)]_1, [1]_2)$$

ii. check the $a(X) \cdot b(X) - c(X) = 0$ relation:

$$e([a(\tau)]_1, [b(\tau)]_2) \stackrel{?}{=} e([q(\tau)]_1, [z(\tau)]_2) \cdot e([c(s)]_1, [1]_2)$$

which ensures that $\exists\ q(X) \in \mathbb{F}[X]$ s.th. $a(X)b(X) - c(X) = q(X)z(X)$.

7 Pinocchio

8 Groth16

9 Powers of tau (trusted setup)

9.1 Main idea

9.2 Participant contribution

In the context of SRS (Structured Reference String), the powers of tau consist of an array containing the scalar multiplication of the generator point by each power of tau, eg. $\tau^0 \cdot G,\ \tau^1 \cdot G,\ \tau^2 \cdot G,\ \tau^3 \cdot G \ldots$

For our case, we'll generate n powers of tau on $\mathbb{G}_1$, and m on $\mathbb{G}_2$. We will denote the combination of both $\mathbb{G}_1$ and $\mathbb{G}_2$ powers of tau by SRS.

Each participant will obtain the previous-participant generated SRS, which is formed by

$$\{[\tau^0]_1, [\tau^1]_1, [\tau^2]_2, \ldots, [\tau^{n-1}]_1\},$$
$$\{[\tau^0]_2, [\tau^1]_2, [\tau^2]_2, \ldots, [\tau^{m-1}]_2\}$$

for simplicity, we will represent it as

$$\{\ [\ \tau^i]_1\ \}_{i=0}^{n-1},\ \{\ [\ \tau^i]_2\ \}_{j=0}^{m-1}$$

Each participant generates their secret random tau τ_p, which to avoid confusion we will denote as $p \in \mathbb{F}_r$. From it, the participant can compute $[p]_2 \in \mathbb{G}_2$.

Then the participant proceeds to update the reference string (SRS), by multiplying each element by p^i:

$$\{\ p^i \cdot [\ \tau^i]_1\ \}_{i=0}^{n-1},\ \{\ p^i \cdot [\ \tau^i]_2\ \}_{j=0}^{m-1}$$

which, by the properties of scalar multiplication of a point, is equivalent to

$$\{\ [\ (p \cdot \tau)^i]_1\ \}_{i=0}^{n-1},\ \{\ [\ (p \cdot \tau)^i]_2\ \}_{j=0}^{m-1}$$

and we denote $p \cdot \tau$ as τ', so we can write the previous expression as

$$\{\ [\ \tau'^i]_1\ \}_{i=0}^{n-1},\ \{\ [\ \tau'^i]_2\ \}_{j=0}^{m-1}$$

which is

$$\{[(\tau \cdot p)^0]_1, [(\tau \cdot p)^1]_1, [(\tau \cdot p)^2]_2, \ldots, [(\tau \cdot p)^{n-1}]_1\},$$
$$\{[(\tau \cdot p)^0]_2, [(\tau \cdot p)^1]_2, [(\tau \cdot p)^2]_2, \ldots, [(\tau \cdot p)^{m-1}]_2\}$$

Notice that the contributor does not know the previous τ, but can obtain $(\tau \cdot p)^i$.

9.3 Proof of correct computation

The participant will compute the proof, which consists of

$$\pi = ([\tau']_1,\ [p]_2)$$

which is equivalent to $\pi = (\tau' \cdot G,\ p \cdot H) = ((p \cdot \tau) \cdot G,\ p \cdot H)$

In the next section we will see how this is used to verify the contribution correct computation.

9.4 Contribution verification

This is the interesting part. We want to check that the new SRS is well computed from the previous SRS together with the new tau. We want to avoid accepting contributions that do not follow the powers of tau structure, or that contain empty elements or that do not derive from the previous SRS.

Following the powers of tau structure means that

$$\tau'^{i+1} = \tau' \cdot \tau'^{i} \quad \forall\ i \in [1, n-1]$$

And the relation between the new SRS and the previous SRS is

$$\tau' = \tau \cdot p$$

where τ is the secret element from the previous SRS and p is the secret one from the new SRS, resulting in the combined τ'.

Now, we're interested into how we can check these relations in the context of the SRS elements. A new contribution can be verified as follows:

1. Check that elements of the new SRS are non-empty, non-zero and in the correct prime order subgroups.
2. Get the proof ($\pi = ([\tau']_1, [p]_2)$), and confirm that the 1st element is equal to the element in position 1 $[\tau'^1]_1$ of the new SRS (which corresponds to τ' powered to 1).
3. Check that τ' (the new SRS) is correctly related to τ (the previous SRS).

   To check this, we rely on the pairing properties and we use the proof value generated by the contributor, which consists of $[p]_2$

   $$e([\tau]_1, [p]_2) \stackrel{?}{=} e([\tau']_1, [1]_2)$$

   We can see how this holds, as $\tau' = p \cdot \tau$, and by the bilinear property of the pairing (see [3]) we have $e(a \cdot G, b \cdot H) = e((a \cdot b) \cdot G, H)$, which in our notation is represented as

   $$e([a]_1, [b]_2) = e([a \cdot b]_1, [1]_2)$$

   by applying this to our case we can see that:

   $$e([\tau]_1, [p]_2) = e([\tau \cdot p]_1, [1]_2) = e([\tau']_1, [1]_2)$$

4. Check if the new SRS follows the powers of tau structure:

   $$e([\tau'^{i}]_1, [\tau']_2) \stackrel{?}{=} e([\tau'^{i+1}]_1, [1]_2) \quad \forall\ i \in [1, n-1]$$

   $$e([\tau']_1, [\tau'^{j}]_2) \stackrel{?}{=} e([1]_1, [\tau'^{j+1}]_2) \quad \forall\ i \in [1, m-1]$$

   Which, again, by the bilinarity property, we can see that

   $$e([\tau'^{i}]_1, [\tau']_2) = e([\tau'^{i+1}]_1, [1]_2)$$

   is enforcing that $\tau'^{i} \cdot \tau'^{1} = \tau'^{i+1}$.

   And the same for

   $$e([\tau']_1, [\tau'^{j}]_2) = e([1]_1, [\tau'^{j+1}]_2)$$

   which checks that $\tau'^{1} \cdot \tau'^{j} = \tau'^{j+1}$.

   With this check, we're ensuring that each power of tau of the SRS is consistent with the previous one.

References

[1] Bryan Parno, Craig Gentry, Jon Howell, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. Cryptology ePrint Archive, Paper 2013/279, 2013.

[2] Jens Groth. On the size of pairing-based non-interactive arguments. Cryptology ePrint Archive, Paper 2016/260, 2016.

[3] `https://github.com/arnaucube/math/blob/master/weil-pairing.pdf`.